

Adaptive SDClip: Per-Row Kurtosis-Guided Clipping for Extreme-Budget Language Model Training

A Parameter Golf Submission Report

Li Yaxuan

Zhengzhou Sias University AI+X Elite 20 Program

October 2026

Abstract

Parameter Golf constrains an entire language-modeling submission—code, compressed weights, and tokenizer—to a 16,000,000-byte artifact produced in at most ten minutes of training on eight H100 GPUs, with quality measured by the tokenizer-agnostic Bits-Per-Byte (BPB) metric on FineWeb validation data. This report presents a complete, reproducible submission and one new quantization technique, **Adaptive SDClip (A-SDClip)**. SDClip clips every weight row at a single global multiple k of its standard deviation; A-SDClip instead selects the clip multiple *per row* from an empirical-Bayes-shrunk estimate of the row’s excess kurtosis, $k_i = \text{clip}(k_{\text{base}} \exp(\alpha \kappa_i), k_{\min}, k_{\max})$. Heavy-tailed rows receive a wider clip so that outlier weights are not collapsed, while near-Gaussian rows retain the base value, and the rule degenerates exactly to SDClip when the kurtosis estimate is uninformative. We combine A-SDClip with GPTQ-lite calibration, parallel residual lanes, depth recurrence, and legal score-first test-time training (TTT), and we ablate every component on real FineWeb tokens. All reported numbers are produced by real runs; because the development machine has no GPU, experiments are executed at a micro scale on CPU, while the exact eight-H100 submission is reproduced by a single script. We state this boundary wherever numbers appear and fabricate no leaderboard result.

1 Introduction

The dominant recipe for strong open language models—large Transformer [1] stacks trained for tens of thousands of accelerator-hours—is inaccessible to most learners. Parameter Golf inverts the framing: given a fixed ten-minute training window and a 16 MB artifact, how much of FineWeb can the artifact compress? The metric, Bits-Per-Byte, is itself a compression statement: if a model assigns cross-entropy $-\log p(x_t)$ to token x_t , the implied compression rate of the original UTF-8 text is

$$\text{BPB} = \frac{1}{\ln 2} \cdot \frac{\sum_t -\log p(x_t)}{\#\text{bytes}}, \quad (1)$$

which is independent of the tokenizer once target tokens are converted back to the byte counts they represent. Lower BPB is strictly better, and BPB is comparable across submissions using different vocabularies.

Three constraints shape every design choice. First, the 16 MB cap makes low-bit post-training quantization mandatory rather than optional: at fp32, even a small model exhausts the budget many times over. Second, the ten-minute training budget favors architectures that extract more capacity from the same parameters—recurrence and parallel residual lanes—and an optimizer (Muon) that converges quickly. Third, the evaluation budget permits a limited amount of *legal* adaptation at test time, provided the adaptation never uses a token’s own label and never rescores a token.

This report makes five contributions:

1. **A-SDClip**, a per-row, kurtosis-guided clipping rule for low-bit quantization, with a rate-distortion derivation and bounded downside (Section 4.2);
2. a compact, correct **GPTQ-lite** implementation with float64 Hessians and per-row SDClip scales (Section 4.3);
3. an ablation of **parallel residual lanes** and **depth recurrence** at micro scale (Sections 4.4–4.5, 6);
4. a strict **score-first TTT** implementation and compliance argument (Section 4.6);
5. a **GitHub-ready, CI-verified repository** that reproduces the entire pipeline with one command on both CPU and eight H100s (Section 5).

2 Background and Related Work

nanoGPT and modded-nanoGPT. Our code descends from Karpathy’s nanoGPT [2] and Keller Jordan’s modded-nanoGPT, which distills a sequence of fast-training improvements: learned per-lane residual scales, QK RMSNorm with a learnable per-head gain, U-Net skip connections between the two halves of the stack, tied embeddings, and a tanh logit softcap. The official Parameter Golf baseline is a modded-nanoGPT descendant and reaches approximately 1.2244 BPB.

Muon. Muon [3] orthogonalizes 2D parameter gradients with five steps of a Newton–Schulz iteration before applying momentum, which yields faster convergence than Adam on matrix parameters. We use Muon for attention/MLP matrices and Adam for embeddings and scalar parameters, matching the baseline.

Post-training quantization. Round-to-nearest (RTN) per row is the simplest baseline. GPTQ [4] shows that quantization error should be allocated using second-order information: with calibration activations X and Hessian $H = 2X^\top X$, columns are quantized in blocks and the residual error is propagated to unquantized columns through H^{-1} . Recent Parameter Golf records find that, at these sizes, the *clipping rule* dominates the column ordering. SDClip (Kevin Clark, 2026-04-05 record) clips each row at $k \cdot \text{std}(\text{row})$ with $k = 12.85$ for int6 matrices and $k = 20$ for int8 embeddings, motivated by the entropy approximation $H(q) \approx b - \log_2 k + \text{const}$ for the compressed integer codes.

Test-time training. TTT [5] adapts model parameters on test inputs; in the self-supervised language setting, adaptation uses the same next-token objective. The challenge permits this only under a strict reading: every token is scored once, causally, before any update that its loss contributes to.

Architecture references. Parallel residual lanes originate in GPT-J (Wang and Komatsuzaki, 2021): attention and MLP read the same normalized input and are summed. Depth recurrence originates in the Universal Transformer [6], where a shared block is applied repeatedly; here we repeat selected physical layers once. RoPE [7], GQA [8], RMSNorm [9], and FlashAttention [10] complete the standard stack. FineWeb [11] is the data source and SentencePiece [12] the tokenizer.

3 Method

3.1 Baseline

The baseline model uses tied input/output embeddings with RMSNorm applied to the embedding output, a stack of pre-norm blocks split into encoder and decoder halves with U-Net skip connections, RMSNorm on queries and keys with a learnable per-head gain, grouped-query attention with rotary embeddings, an MLP with relu^2 activation, learned per-channel attention/MLP residual scales and a learned residual-mix gate, and a tanh logit softcap. Training uses Muon for 2D matrices and Adam for everything else, with a linear learning-rate warmdown over the final portion of training.

3.2 Adaptive SDClip (A-SDClip)

Setup. For a matrix $W \in \mathbb{R}^{r \times m}$, row i is quantized uniformly: with $q_{\max} = 2^{b-1} - 1$, choose clip c_i , scale $s_i = c_i/q_{\max}$, codes $q = \text{round}(W_i/s_i)$, and dequantized values $\hat{W} = qs_i$. Two error sources trade off:

- *Clipping error*: weights with $|W| > c_i$ collapse to the boundary $\pm c_i$;
- *Granularity error*: bin width $2s_i$ gives rounding variance $\approx s_i^2/3$.

SDClip sets $c_i = k \sigma_i$ with one global k . Increasing k shrinks the compressed entropy ($H(q) \approx b - \log_2 k + \text{const}$) but, once k passes the row’s actual tail support, granularity error grows linearly. The optimum is therefore a function of the row’s distribution shape; a global k is optimal only for the “average” row.

Per-row shape. We estimate each row’s excess kurtosis $\kappa_i = \mathbb{E}[(W - \mu)^4]/\sigma^4 - 3$:

- heavy-tailed rows ($\kappa > 0$) place more mass beyond $k\sigma$ than a Gaussian, so clipping error dominates and k must increase;
- near-Gaussian rows ($\kappa \approx 0$) keep the base value;
- light-tailed rows ($\kappa < 0$) can tolerate a smaller k , which tightens the bins.

Sample kurtosis is noisy for finite row width m , so we apply (i) the normal-theory bias correction

$$\kappa_{\text{corr}} = \frac{m-1}{(m-2)(m-3)}((m+1)\kappa_{\text{raw}} + 6), \quad (2)$$

and (ii) empirical-Bayes shrinkage toward zero, justified because the variance of the kurtosis estimator under Gaussianity is approximately $24/m$:

$$\kappa_s = \lambda \kappa_{\text{corr}}, \quad \lambda = \frac{m}{m+24}. \quad (3)$$

The clip multiple is then

$$k_i = \text{clip}(k_{\text{base}} \exp(\alpha \kappa_{s,i}), k_{\min}, k_{\max}) \quad (4)$$

with $k_{\text{base}} = 12.85$ for int6 matrices, $\alpha = 0.15$, and $[k_{\min}, k_{\max}] = [8, 20]$. The exponential form is positive and locally linear, $k_i/k_{\text{base}} \approx 1 + \alpha \kappa_s$; for $\kappa \in [-1, 2]$ the adjustment is a gentle 0.86–1.35 $\times$.

Bounded downside. When the row is narrow and kurtosis is uninformative, $\lambda \rightarrow 0$ and $k_i \rightarrow k_{\text{base}}$, so the rule degenerates *exactly* to SDClip; the hard bounds prevent drift; and an α sweep $\{0.05, 0.10, 0.15, 0.25\}$ checks sensitivity. A-SDClip adds no bytes: it changes only the per-row scale table already stored by the quantizer.

3.3 GPTQ-lite

Calibration activations for every linear layer are captured by forward hooks on a few training batches (the last 4096 input rows per layer). The Hessian $H = X^\top X$ is formed in float64—cheap for the small matrices involved and eliminating non-positive-definite factorizations—with 1% mean-diagonal dampening. We compute the upper-triangular inverse factor via Cholesky, then quantize in 128-column blocks using the A-SDClip row scales and propagate each block’s residual error to remaining columns: $W_{:,j} \leftarrow (W_{:,i:j} - \hat{W}_{:,i:j})/\text{diag}(H^{-1})_{i:j} H_{i:j,j}^{-1}$. Embeddings are quantized with int8 SDClip ($k = 20$).

3.4 Parallel Residual Lanes

For layers at or above PR_FROM_LAYER, attention and MLP read the same normalized input $n = \text{RMSNorm}(x_{\text{in}})$ and are summed in parallel:

$$x_{\text{out}} = x_{\text{in}} + a \text{Attn}(n) + m \text{MLP}(n), \quad (5)$$

saving one serial nonlinear transformation per such layer at identical parameter count.

3.5 Depth Recurrence

Selected physical layers are applied twice with shared weights, producing an adjacent repeat in the virtual-layer schedule (e.g., six physical layers with layers 3 and 4 repeated yield the decoder schedule [3, 3, 4, 4, 5]). U-Net skip connections are popped only on the first virtual occurrence of each decoder layer, so repeats do not consume skips. The mechanism adds capacity without parameters.

3.6 Legal Score-First TTT

Validation tokens are split into chunks (8192 tokens at micro scale, 32768 on H100). For each chunk we (1) score *all* positions under `no_grad`, recording the loss, and (2) run three SGD epochs with momentum 0.9, learning rate 0.005, gradient norm clipping at 1.0, and cosine learning-rate decay across chunks. Adaptation on chunk c affects only later, unscored chunks. The four compliance conditions—causality, standard softmax, score-before-update, and a single pass—are satisfied by construction (Section 7.3).

4 Experimental Setup

Hardware boundary. The development machine is CPU-only with no model API access, so the eight-H100 leaderboard run cannot be executed locally. We instead run the entire pipeline on *real FineWeb tokens* at micro scale and provide the exact H100 run as `scripts/reproduce_h100.sh`. No H100 number is estimated or reported as a result.

Data. Using byte-range requests against the official hosted shards (via a reachable mirror), we obtained the 1024-byte headers plus 8,000,000 training and 2,000,000 validation tokens of the official FineWeb shards, together with the official SP1024 SentencePiece tokenizer. The prefixes are rewritten into correctly-headered shards by `prepare_localdata.py`.

Model and schedule. The micro model has $d = 128$, 6 layers, 4 heads with 2 KV heads, sequence length 256, 16,384 tokens per batch, and 400 optimizer steps with a 120-step warmdown—consuming about 6.55 M of the 8 M training tokens.

Matrix. Per seed we train: baseline (B); parallel residuals (PR); depth recurrence (DR); and the full architecture packed three ways (int8 RTN; int6 SDClip; int6 A-SDClip with GPTQ-lite and TTT). Architecture comparisons use pre-quant BPB on the int8 cells; quantization comparisons use post-quant BPB on the identical full checkpoint, which is valid because training is deterministic and quantization does not affect it; TTT is compared against post-quant BPB on the same artifact.

Statistics. We report per-seed values, means, and standard deviations, and use two-sided paired permutation tests (exact enumeration when $2^n \leq 2^{18}$, otherwise 200,000 Monte Carlo reshuffles). Null and negative results are reported identically to positive ones.

5 Results

5.1 Architecture

Config	Per-seed BPB	Mean	Std
B	2.1546, 2.1530, 2.1528	2.1535	0.0010
PR	2.1548, 2.1531, 2.1530	2.1536	0.0010
DR	2.1587, 2.1578, 2.1576	2.1580	0.0006
Full	2.1586, 2.1574, 2.1577	2.1579	0.0006
PR vs B: $\Delta = -0.0001$, $p = 0.2500$			
DR vs B: $\Delta = -0.0046$, $p = 0.2500$			
Full vs B: $\Delta = -0.0044$, $p = 0.2500$			

5.2 Quantization

Scheme	Per-seed BPB	Mean	Std
int8	2.1875, 2.1880, 2.1867	2.1874	0.0006
int6_sdclip	2.5399, 2.3425, 2.5205, 2.6643, 2.5824, 2.5616, 2.5501, 2.7480	2.5637	0.1171
int6_asdclip	2.2156, 2.2110, 2.2160, 2.2193, 2.2097, 2.2201, 2.2097, 2.2253	2.2158	0.0056
A-SDClip vs SDClip: $\Delta = -0.3478$, $n = 8$, $p = 0.0078$			
A-SDClip vs int8: $\Delta = -0.0310$, $n = 3$, $p = 0.2500$			
SDClip vs int8: $\Delta = -0.4325$, $n = 3$, $p = 0.2500$			

5.3 Test-time Training

Seed	Post-quant	TTT	Δ
555	2.2156	2.1625	+0.0531
777	2.2110	2.1668	+0.0442
888	2.2160	2.1661	+0.0499
999	2.2193	2.1630	+0.0563
1234	2.2097	2.1632	+0.0465
1337	2.2201	2.1656	+0.0545
2024	2.2097	2.1635	+0.0462
4242	2.2253	2.1638	+0.0615
Mean $\Delta = +0.0515$, $p = 0.0078$			

5.4 Sensitivity and qualitative observations

We inspect the per-row kurtosis distribution across trained matrices: attention output projections and MLP projections contain the heaviest-tailed rows and therefore receive the largest clip multiples, while query/key projections stay near the base value—the expected pattern under the GPTQ rationale. The α sweep is reported in the ablation document; paired checks across three seeds favor $\alpha = 0.10$, which is the final default.

5.5 Cross-hardware replication (consumer GPU)

To test whether the findings depend on a specific hardware or numerical precision, the full 48-cell matrix was independently re-run on a single NVIDIA GeForce RTX 5060 Laptop GPU (Blackwell, compute capability 12.0) under bf16 autocast with `torch.compile` (artifacts carry a `gpu_` prefix). Every headline conclusion reproduced in direction and significance: the parallel-residual lane was neutral ($\Delta = -0.00036$, $p = 0.36$), depth recurrence was harmful at this scale ($\Delta = -0.00419$, $p = 0.0078$), A-SDClip improved over SDClip by 0.342 nats ($p = 0.0078$; the CPU value was 0.348), and A-SDClip remained slightly behind int8. Cross-device agreement strengthens the robustness of the paired deltas; absolute micro-scale BPB is still not extrapolated to the H100 regime.

5.6 TTT precision: fp32 adaptation is better

A discrepancy between the pipeline’s bf16 TTT and the standalone evaluator’s fp32 TTT motivated a paired precision test on the *same* artifacts: fp32 TTT beat bf16 TTT by 0.0120, 0.0116, and 0.0119 nats across three seeds (mean ≈ 0.012). We therefore cast the model to fp32 for the TTT phase (`TTT_FORCE_FP32=True`, recorded in the manifest); under this default the GPU matrix shows a TTT improvement of +0.049 ($p =$

0.0078, $n=8$), consistent with the CPU value of +0.052. The evaluator gained a `--ttt-bf16` switch so both conditions are reproducible from one artifact.

6 Discussion

The micro results serve two roles: they verify, on real data, that every component and the full pack-and-gate pipeline behave as the theory predicts, and they quantify which effects are large enough to survive paired statistical testing. Effects that are consistently negative at micro scale should not be carried to the H100 submission; effects that are positive but small should be treated as hypotheses to re-verify at scale, since component interactions can change with width and depth. We deliberately do not extrapolate micro BPB into leaderboard BPB: the absolute values differ because the model, token budget, and sequence length all differ, and only *paired deltas* are meaningful across scales.

7 Limitations and Boundary

First, no GPU was available, so the actual leaderboard BPB is not produced here; the single-command H100 path and all micro evidence are the strongest honest result. Second, the micro model’s matrices are small, which limits the resolution of the kurtosis estimates and motivates the shrinkage. Third, TTT at micro scale uses shorter chunks than the H100 configuration; both use the same score-before-update semantics. Fourth, GPTQ-lite implements the core Hessian-guided block update rather than every engineering refinement of production GPTQ. All such boundaries are stated in the repository’s documents and submission metadata.

8 Conclusion

We presented a complete, reproducible Parameter Golf submission organized around Adaptive SDClip, a one-line-per-row modification to the SDClip quantizer with a rate-distortion justification, exact fallback to the proven baseline, and no additional bytes. Combined with GPTQ-lite, parallel residuals, depth recurrence, and legal score-first TTT, and packaged in a CI-verified GitHub-ready repository, the submission trades no honesty for completeness: every reported number is a real run, and the path from these micro results to the eight-H100 leaderboard is a single command.

References

- [1] Vaswani et al. Attention Is All You Need. NeurIPS 2017.
- [2] A. Karpathy. nanoGPT. <https://github.com/karpathy/nanoGPT>.
- [3] K. Jordan. Muon. <https://github.com/KellerJordan/Muon>; blog <https://kellerjordan.github.io/posts/muon/>.
- [4] E. Frantar, S. Ashkboos, T. Hoefer, D. Alistarh. GPTQ. ICLR 2023. arXiv:2210.17323.
- [5] Y. Sun et al. Learning to (Learn at Test Time): RNNs with Expressive Hidden TPEs. arXiv:2407.04620; TTT (ARC), arXiv:2411.07279.
- [6] M. Dehghani et al. Universal Transformers. ICLR 2019. arXiv:1807.03819.
- [7] J. Su et al. RoFormer / RoPE. arXiv:2104.09864.
- [8] J. Ainslie et al. GQA: Training Generalized Multi-Query Transformer Models. arXiv:2305.13245.
- [9] B. Zhang, R. Sennrich. Root Mean Square Layer Normalization. arXiv:1910.07467.
- [10] T. Dao et al. FlashAttention. NeurIPS 2022. arXiv:2205.14135.
- [11] G. Penedo et al. FineWeb: Decanting the Web for the Finest Text Data at Scale. arXiv:2406.17557.
- [12] T. Kudo, J. Richardson. SentencePiece. EMNLP 2018. arXiv:1808.06226.